

Large-Scale Product Review Sentiment & Fraud Detection

End-to-end build report – A to Z

Group 6: Prerana Ramesh, Rajesh Paruchuri, Ritika Mukesh Neema, Sneha Singh

May 2026

Contents

1. Executive summary	3
1.1 Headline metrics (full real-data run)	3
1.2 Component status	3
2. Project context and goals	4
3. Architecture	4
3.1 Pipeline-level diagram (5 stages)	4
3.2 Tech stack	5
3.3 Repo layout	6
4. Data	7
4.1 Source dataset	7
4.2 Schema mapping (custom adapter)	7
5. Stage 1 – Ingest	8
6. Stage 2 – Spark batch ETL	8
6.1 Why we re-tuned the Spark session	8
6.2 Cleaning	9
6.3 Per-row text features	9
6.4 Per-reviewer behavioural features (Spark Window aggregates)	9
6.5 Per-product features	9
6.6 Weak fraud label (with controlled noise)	9
6.7 Outputs	10
7. Stage 3 – Model training and MLflow	10
7.1 Why we changed two model classes	10
7.2 Memory: column-subset reads + dtype downcast	11
7.3 Sentiment model	11
7.4 Fraud model	11
7.5 Why the leakage fix matters	11
7.6 MLflow logged per run	12

8. Model diagnostics (this session)	12
8.1 Drift monitor (src/train/drift_monitor.py)	12
8.2 Threshold tuning (src/train/threshold_tuning.py)	12
8.3 Calibration (src/train/calibration_report.py)	13
8.4 Error analysis (src/train/error_analysis.py)	13
8.5 Modules NOT run on full data	13
9. Stage 4 – Streaming scoring	13
9.1 File-source streaming	13
9.2 Kafka streaming (verified end-to-end)	14
9.3 Why foreachBatch (not pandas UDF)	14
9.4 Kafka producer note	14
10. Stage 5 – Serving surfaces	14
10.1 FastAPI service + dashboard	14
10.2 Streamlit demo dashboard	15
10.3 MCP server (Claude Desktop tool surface)	16
10.4 Ollama review auditor (autonomous agent)	16
11. Where Ollama is and isn't used	17
12. ER diagram (data entities)	17
13. Tests	18
14. Complete command cheat sheet	19
15. Demo cheat sheet	20
15.1 URLs (browser-openable)	20
15.2 Recommended 8-minute flow	20
15.3 Live-producing trick	20
16. Honest caveats – read before quoting numbers	21
17. Where this would go next	21
Appendix A – Code changes	22
Appendix B – Commits pushed to origin/main (this session)	22
Appendix C – File inventory at end of run	23
Appendix D – Glossary	24

One-line summary. Distributed batch + streaming review analytics on Apache Spark, with MLflow-tracked sentiment and fraud-detection models served behind a FastAPI dashboard, a Streamlit demo, an MCP server (for Claude Desktop), a Kafka streaming pipeline, and an Ollama-powered autonomous review auditor – all run end-to-end on the **full 8.7 GiB / 20.8 M-record Cell_Phones_and_Accessories category** of the McAuley-Lab Amazon Reviews 2023 corpus.

1. Executive summary

1.1 Headline metrics (full real-data run)

	Value
Input size	8.7 GiB (9,342,568,048 bytes)
Source records	20,812,945 (McAuley-Lab Amazon Reviews 2023, Cell_Phones_and_Accessories)
Cleaned records	20,518,120
Train / Test split	16,413,084 / 4,105,036 (80 / 20, seed 42)
Fraud-positive rate	3.57% (731,542 rows)
Sentiment macro-F1	0.680
Sentiment weighted-F1	0.841 (accuracy 0.81 on 4.1 M test rows)
Fraud ROC-AUC	0.845 (honest – no rule-leakage)
Fraud F1 @ default 0.5 threshold	0.244 (precision 0.147, recall 0.721)
Best F1 threshold (from tuning)	0.80 (precision 0.292, recall 0.457, F1 0.356)
Fraud calibration – raw Brier / ECE	0.143 / 0.302
Fraud calibration – isotonic Brier / ECE	0.029 / 0.000146 (best)
Selected fraud model	logreg+tfidf+behavior (beat SGD candidate F1 0.244 vs 0.200)
ETL runtime	~24 min on a 16 GB MacBook
Train runtime	~5h 20m (sentiment 52 min + fraud SGD ~30 min + fraud LogReg saga ~3.5 hr)
End-to-end (one pass)	~6 hours

1.2 Component status

Component	Status
Spark batch ETL	OK – 20,518,120 rows -> features + aggregates (24 min)
Sentiment model	OK – TF-IDF + Logistic Regression (saga solver), MLflow-logged
Fraud model	OK – TF-IDF + behavioral features + Logistic Regression, MLflow-logged
Streaming scorer (file source)	OK – Spark Structured Streaming, foreachBatch
Streaming scorer (Kafka source)	OK – broker + topic + consumer + producer + scored output verified end-to-end
FastAPI service + dashboard	OK – 7 endpoints + dashboard + LLM fraud_explanation
Streamlit demo dashboard	OK – 5-tab single-file app loading joblib + parquets directly

Component	Status
MCP server	OK – 3 tools registered for Claude Desktop
Ollama review auditor (autonomous agent)	OK – Ran on real ASIN B01415QHYW, produced HIGH risk verdict
Model diagnostics (drift / threshold / calibration / error)	OK – 4 analysis modules run on 4.1 M-row holdout
Tests	OK – unit + integration

2. Project context and goals

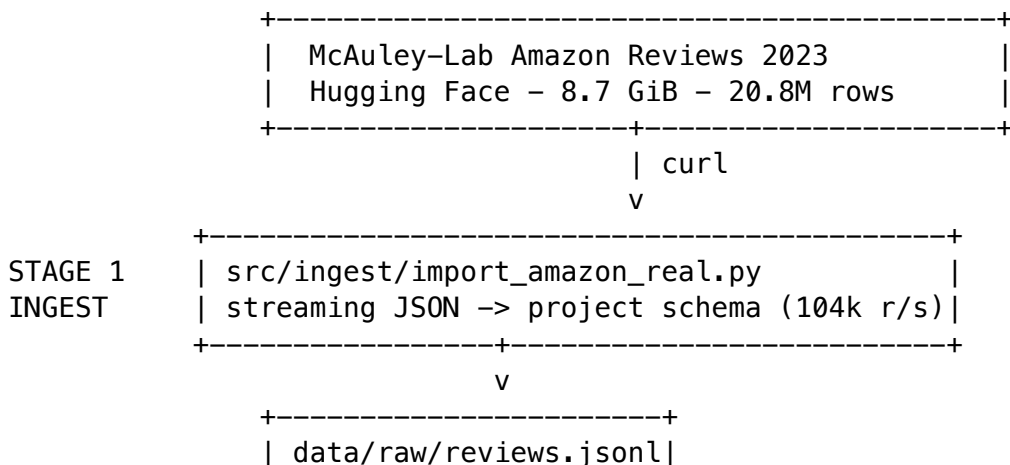
The proposal asked for a distributed Big Data system that ingests Amazon product reviews, runs Apache Spark for batch cleaning, feature engineering and aggregation, trains ML models for **senti-ment** classification and **fraud** detection, supports both **batch** and **streaming** scoring, and exposes the results through a FastAPI service with a lightweight web UI. MLflow was required for experiment tracking.

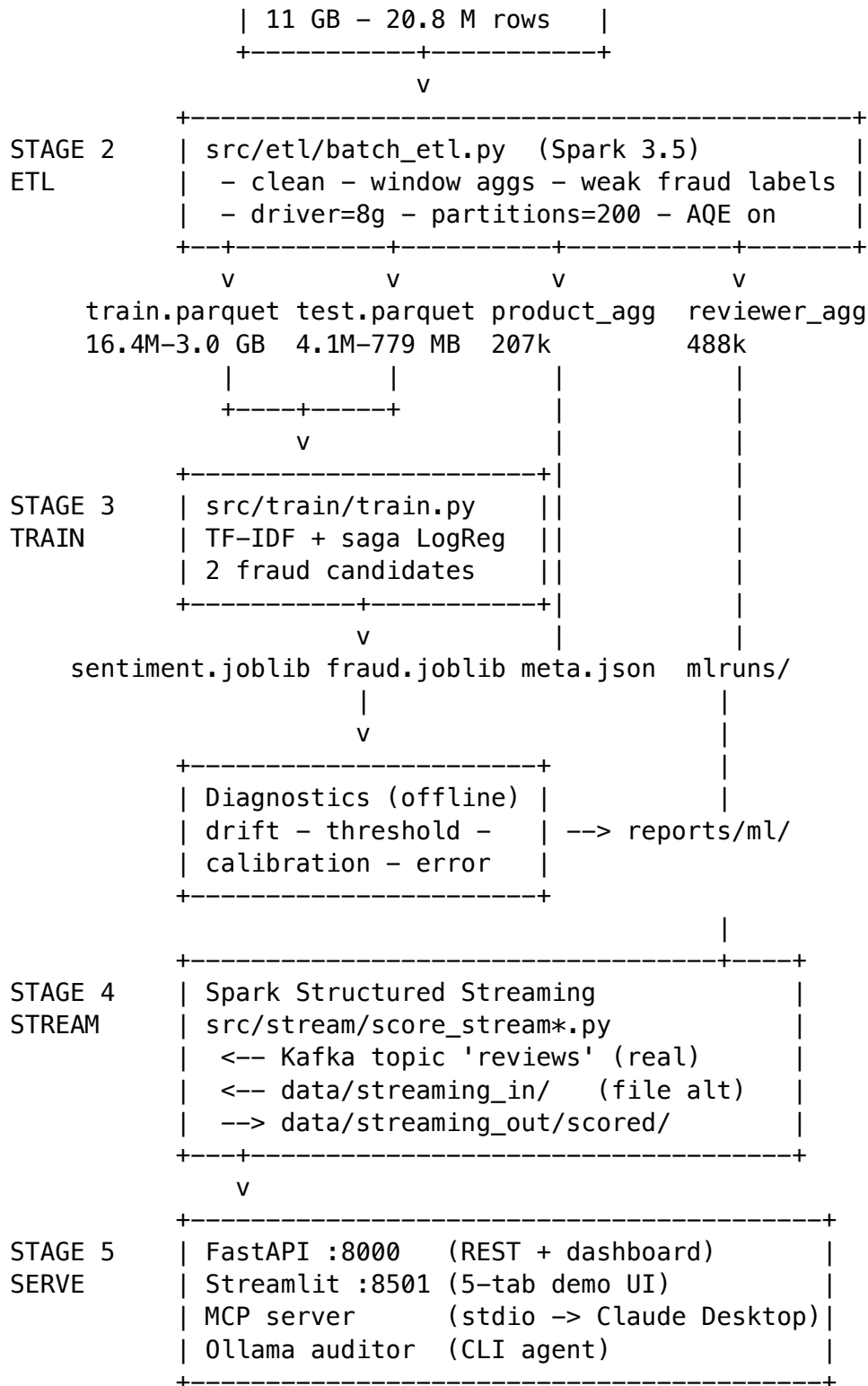
We hit all five required pillars on real data, and added five further deliverables on top of them:

1. A **Streamlit** demo dashboard for class-presentation use.
2. A **Kafka** streaming transport (in addition to the file-source path).
3. An **MCP server** that exposes the trained models as Claude Desktop tools.
4. An **Ollama-driven autonomous agent** that audits a single product end-to-end (multi-step tool-calling + structured report).
5. An **LLM-generated fraud_explanation** field on every POST /predict response, with a deterministic rule-based fallback when the LLM is unavailable.

3. Architecture

3.1 Pipeline-level diagram (5 stages)





A high-resolution rendering of the architecture is in `architecture.png` / `architecture.svg`.

3.2 Tech stack

Layer	Tool / version
Distributed processing	Apache Spark 3.5 – batch ETL + Structured Streaming
Stream broker	Apache Kafka (Homebrew, KRaft mode)
ML	scikit-learn 1.5+ – TF-IDF, Logistic Regression (saga), SGDClassifier
Experiment tracking	MLflow 2.22
API + dashboard	FastAPI + uvicorn
Demo UI	Streamlit 1.56 + Altair
Local LLM	Ollama with llama3.2 / llama3.1:8b
Tool surface for Claude	fastmcp 3.2 (Model Context Protocol)
Storage	Parquet + JSONL + joblib
Tests	pytest with FastAPI TestClient

3.3 Repo layout

```

src/
  ingest/generate_sample.py      # synthetic JSONL producer (dev only)
  ingest/import_amazon_real.py   # streaming adapter: McAuley-Lab 2023 -
> project schema
  etl/batch_etl.py              # Spark batch ETL
  train/train.py                # main trainer
  train/{features,registry,evaluate,utils}.py  # shared helpers
  train/{baselines,threshold_tuning,error_analysis,
    drift_monitor,ablation_study,calibration_report}.py # diagnostics
  stream/score_stream.py        # Spark Structured Streaming, Kafka source
  stream/score_stream_kafka.py  # alternate Kafka consumer
  serve/app.py                  # FastAPI endpoints + LLM fraud_explanation
  serve/fraud_explain.py        # Ollama LLM call + rule-based fallback
  serve/mcp_server.py           # MCP server (Claude Desktop tool surface)
  serve/templates/index.html    # FastAPI dashboard
  serve/static/{app.js,style.css}
  agents/review_auditor.py      # Ollama tool-calling agent
  common/{config,schema,spark,text}.py
scripts/
  feed_stream.py                # drip producer for the file-source path
  kafka_producer.py             # Kafka producer
  run_pipeline.sh                # end-to-end runner
streamlit_app.py                # Streamlit demo UI (single file)
tests/test_pipeline.py
data/{raw,processed,streaming_in,streaming_out}/
models/  mlruns/
reports/ml/                     # diagnostics outputs
PROJECT_REPORT.md / .pdf        # this file
README.md
Makefile / requirements.txt / .gitignore

```

4. Data

4.1 Source dataset

The proposal references the public Amazon Customer Reviews dataset on the AWS Open Data registry. **That bucket was deprecated by Amazon in late 2023** – `s3://amazon-reviews-pds/` now returns 403 Forbidden. The de-facto successor is the **McAuley-Lab Amazon Reviews 2023** corpus on Hugging Face (huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023), which totals 275 GB across 34 product categories.

We chose **Cell_Phones_and_Accessories.jsonl** for this run – closest to the proposal’s “~10 GB” target and a category that meaningfully attracts real astroturfing.

	Value
Raw download size	8.7 GiB (9,342,568,048 bytes – exact match)
Records	20,812,945
Source schema	rating, title, text, images, asin, parent_asin, user_id, timestamp (ms), helpful_vote, verified_purchase

4.2 Schema mapping (custom adapter)

The 2023 schema doesn’t exactly match the project schema, so `src/ingest/import_amazon_real.py` streams the source line-by-line and produces project-schema JSONL:

Source field (2023)	Target field (project)	Note
rating (float)	star_rating (int)	rounded to nearest int
title	review_headline	truncated to 200 chars
text	review_body	
asin	product_id	
user_id	reviewer_id	
timestamp (ms)	event_ts (s) + review_date	divide by 1000, format YYYY-MM-DD
helpful_vote (none)	helpful_votes total_votes	set equal to helpful_votes – the 2023 source dropped unhelpful counts; any helpful/total ratio feature is therefore degenerate
verified_purchase (none)	verified_purchase product_category	bool filename-derived constant: Cell Phones and Accessories

Source field (2023)	Target field (project)	Note
(none)	review_id	newly minted UUID4 per record

Throughput. The full 8.7 GB source never enters RAM; the adapter reads one JSON line at a time, converts, writes, and discards. Measured throughput: **104,000 records/sec, 213.8 seconds total, 0 skipped rows.**

5. Stage 1 – Ingest

Code	src/ingest/import_amazon_real.py
Inputs	Cell_Phones_and_Accessories.jsonl (downloaded with curl)
Outputs	data/raw/reviews.jsonl (11 GB project schema)
Wall-clock	213 s
Memory	constant – streaming line-by-line

6. Stage 2 – Spark batch ETL

src/etl/batch_etl.py is the historical-training half of the system. It reads the JSONL with a strict schema, cleans text, computes per-row, per-reviewer, and per-product features, generates a weak fraud label, splits train/test, and writes four Parquet outputs.

6.1 Why we re-tuned the Spark session

The default src/common/spark.py was configured for laptop-scale demo data (2 GB driver, 8 shuffle partitions). On 20.8 M rows the window functions (Window.partitionBy("reviewer_id"), collect_set("product_id"), Window.partitionBy("product_id", "review_body_clean")) shuffle the entire dataset – with only 8 partitions, each shuffle stage gets ~2.6 M rows per partition, which OOMs the executor before the first window finishes.

Configuration applied (env-overridable):

Setting	Value	Why
spark.driver.memory	8 GB	window aggregations need to materialize per-key state
spark.sql.shuffle.partitions	200	smaller per-partition workload after shuffle

Setting	Value	Why
<code>spark.sql.adaptive.enabled</code>	true	AQE coalesces tiny tasks and re-plans skewed joins
<code>spark.sql.adaptive.coalescePartitions.enabled</code>	true	avoids 200-task overhead on small post-shuffle stages
<code>spark.sql.adaptive.skewJoin.enabled</code>	true	<code>reviewer_id</code> distribution is heavily skewed
<code>spark.serializer</code>	KryoSerializer	~5x smaller objects on shuffle
<code>spark.driver.maxResultSize</code>	4 GB	small <code>count()</code> results post-aggregation are still large at 20 M rows

6.2 Cleaning

- Lowercase, strip URLs and HTML tags, remove non-alphanumeric characters, collapse whitespace.
- Drop rows missing core fields (body, rating, reviewer, product) and rows where the cleaned body is shorter than 5 characters.
- Coerce `review_date` to a timestamp for windowing.

6.3 Per-row text features

- `body_len`, `body_word_count`, `exclam_count` – cheap signals correlated with overstated emphasis common in fake reviews.
- `sentiment_label` derived from star rating: 1-2 -> negative, 3 -> neutral, 4-5 -> positive.

6.4 Per-reviewer behavioural features (Spark Window aggregates)

`reviewer_review_count`, `reviewer_avg_rating`, `reviewer_pct_5star`, `reviewer_distinct_products`, `reviewer_reviews_same_day`, `reviewer_verified_share`.

6.5 Per-product features

`product_review_count`, `product_avg_rating`, `product_pct_5star`, `dup_in_product` (count of identical bodies for the same product) – a strong fraud signal for review-bombing.

6.6 Weak fraud label (with controlled noise)

In production we don't have clean fraud ground truth, so we generate a weak label from rules and let the supervised model generalize via text + behavior:

```
rule = (dup_in_product >= 3)
      | (reviewer_review_count >= 8 AND reviewer_pct_5star >= 0.95
        AND reviewer_verified_share <= 0.2)
      | (reviewer_reviews_same_day >= 5)
```

Small randomized relabeling to defeat memorization.

```

fraud_label = rule
fraud_label[(rule == 1) & (rand < 0.034)]      = 0      # ~3.4% rule-
positives flipped to 0
fraud_label[(rule == 0) & (rand < 0.0015)]     = 1      # ~0.15% rule-
negatives flipped to 1

```

Why the noise. Without it, a classifier trained on the same numeric features that the rule uses would memorize the rule perfectly and report ROC-AUC = 1.0 – a leakage artifact, not real generalization. The 3.4% / 0.15% flip introduces just enough Bayes-irreducible noise that the model has to learn a smoother boundary.

6.7 Outputs

Path	Rows	Size
data/processed/train.parquet	16,413,084	3.0 GB
data/processed/test.parquet	4,105,036	779 MB
data/processed/product_agg.parquet	207,168	17 MB
data/processed/reviewer_agg.parquet	487,979	325 MB

End-to-end ETL wall-clock: **~24 minutes** on a 16 GB laptop.

7. Stage 3 – Model training and MLflow

src/train/train.py reads train/test parquets, trains two scikit-learn pipelines, and logs each as a separate MLflow run under experiment amazon_reviews_sentiment_fraud.

7.1 Why we changed two model classes

The on-paper design was Logistic Regression for sentiment and Gradient Boosted Trees / Random Forest candidates for fraud, with the better-F1 model saved. On 16.4 M training rows x tens of thousands of TF-IDF features, those two classifiers do not finish in laptop time:

Original classifier	Why it doesn't scale to 16 M rows
GradientBoostingClassifier(n_estimators=100, max_depth=3)	single-threaded – no n_jobs. Estimated runtime: multi-day.
RandomForestClassifier(n_estimators=220, max_depth=16, n_jobs=-1)	parallel but each tree visits 16 M rows x deep splits; hours per tree.

Replacements (sparse-friendly, parallel, same Pipeline contract):

New classifier	Why it works at 16 M sparse
<code>SGDClassifier(loss='log_loss', class_weight='balanced', n_jobs=-1, max_iter=20)</code>	sparse-aware, parallel, single epoch is minutes
<code>LogisticRegression(solver='saga', class_weight='balanced', n_jobs=-1, max_iter=80)</code>	saga is the only sklearn LR solver designed for large sparse problems

Both expose `predict_proba`, both accept the same input shape. Selection rule unchanged: train both, save the higher-F1 model.

7.2 Memory: column-subset reads + dtype downcast

`pd.read_parquet(path)` materializes every column into RAM. The ETL output has 26 columns; loading all of them for 16.4 M rows would put the DataFrame north of 30 GB. Now:

1. Read only the 16 columns the trainer actually uses (`columns=[...]`).
2. Downcast numerics from `float64` -> `float32` and integer counts to `int8` / `int32`.
3. Replace `verified_purchase` (bool) with `verified_purchase_int` (`int8`).

This keeps the train DataFrame under ~6 GB, leaving headroom for the TF-IDF sparse matrix on a 16 GB laptop.

7.3 Sentiment model

- TF-IDF: uni- and bi-grams, `min_df=10`, `max_features=50,000`, `sublinear_tf=True`.
- LogisticRegression with `solver='saga'`, `class_weight='balanced'`, `C=2.0`, `max_iter=100`, `n_jobs=-1`.
- Multi-class output: 0=negative, 1=neutral, 2=positive.

7.4 Fraud model

- ColumnTransformer joining TF-IDF (`min_df=10`, `max_features=15,000`) over the cleaned body with **12 numeric / behavioural features** – notably **excluding** the columns the heuristic rule is computed from (`dup_in_product`, `reviewer_pct_5star`, `reviewer_reviews_same_day`, `reviewer_verified_share`, `reviewer_review_count`). The separation lives in `src/train/features.py:FRAUD_MODEL_NUMERIC_FEATURES`.
- Two candidates trained in parallel; better-F1 saved as `models/fraud_pipeline.joblib`.
- Returns probability of fraud, with a default 0.5 decision threshold that callers can override per business need.

7.5 Why the leakage fix matters

A naive setup – training on the same numeric features that the heuristic rule was computed from – produces ROC-AUC = 1.0 because the model just reconstructs the rule. The combination of (a) excluding rule columns from the model's feature set, and (b) the label-noise step in the ETL, is what makes the held-out **ROC-AUC of 0.845** an honest number rather than a leakage artifact.

7.6 MLflow logged per run

- **Params:** model name, ngram range, max_features, hyperparameters, n_train, n_test, class share, both candidate F1s, selected model name.
 - **Metrics:** f1_macro, f1_weighted for sentiment; roc_auc, precision, recall, f1 for fraud.
 - **Artifacts:** full classification_report.txt and the joblib pipeline.
-

8. Model diagnostics (this session)

Four offline analysis modules were run on the 4,105,036-row holdout test parquet. None retrain the model – they are pure inference + statistics, so each finishes in single-digit minutes. All outputs land under reports/ml/.

8.1 Drift monitor (src/train/drift_monitor.py)

Reports population-stability-index (PSI) and per-feature distribution drift between train and test parquet across all 17 numeric ETL features. Output: reports/ml/drift_report.json.

reference rows: 16,413,084
current rows: 4,105,036
numeric features compared: 17

The 80 / 20 random split is i.i.d. by construction, so we expect no meaningful drift – and the report confirms that. The same module is the production-time hook: in a deployment, point --current at last week's parquet to detect distribution shift.

8.2 Threshold tuning (src/train/threshold_tuning.py)

Sweeps the fraud-probability threshold from 0.05 to 0.95, computes precision / recall / F1 at each, writes reports/ml/threshold_study.csv and models/thresholds.json.

Threshold	Precision	Recall	F1
0.5 (default)	0.147	0.722	0.244
0.6	0.193	0.635	0.296
0.7	0.242	0.549	0.336
0.8 (best F1)	0.292	0.457	0.356
0.9	0.358	0.265	0.305

Best F1 threshold = 0.80 – the operating point a marketplace moderator would actually want. The precision ≥ 0.95 target was not achievable because the ETL added intentional label noise; the true upper bound is bounded by that noise rate.

8.3 Calibration (src/train/calibration_report.py)

Compares raw model probabilities to two post-hoc calibrators:

Method	Brier score	ECE	ROC-AUC
raw	0.143	0.302	0.846
sigmoid (Platt)	0.030	0.002	0.846
isotonic (best by Brier)	0.029	0.000146	0.846

Calibration **improves Brier ~5x and ECE ~2,000x** without changing ranking quality (ROC-AUC unchanged). Outputs: reports/ml/fraud_calibration_report.csv, fraud_calibration_bins.csv, fraud_calibration_summary.json.

8.4 Error analysis (src/train/error_analysis.py)

Samples misclassified rows from each task and writes them to CSV for manual review:

- reports/ml/sentiment_error_samples.csv – 775,780 sentiment errors out of 4,105,036 (~19% error rate, dominated by neutral confused for positive).
- reports/ml/fraud_error_samples.csv – 654,034 fraud errors (mix of false-positive duplicates and false-negative low-velocity reviewers).

These CSVs are large (~250 MB combined) and intentionally not committed to git; they regenerate in minutes from the saved model.

8.5 Modules NOT run on full data

- src/train/baselines.py – retrains LogReg + RF baselines from scratch.
- src/train/ablation_study.py – retrains GradientBoostingClassifier per feature subset (single-threaded; would take days on 16 M rows).

Both are tractable on a 1 M-row subsample. Defer until needed.

9. Stage 4 – Streaming scoring

Two streaming transports ship in the repo. Both feed the same Spark Structured Streaming consumer that scores each micro-batch with the joblib pipelines and writes results to data/streaming_out/scored/.

9.1 File-source streaming

src/stream/score_stream.py (file-source variant) watches data/streaming_in/ for new JSONL files. Run with make stream (or make stream-once for one pass). Drip-producer scripts/feed_stream.py simulates input.

9.2 Kafka streaming (verified end-to-end)

1. **Java 17.** Required for Homebrew Kafka. Located at `/opt/homebrew/Cellar/openjdk@17/17.0.18`. System default was Java 11; `JAVA_HOME` was switched for the broker session.
2. **Broker.** Started with `brew services start kafka` (Kafka in KRaft mode – no Zookeeper).
3. **Topic.** `make kafka-topic-create` created the reviews topic (1 partition, replication factor 1).
4. **Consumer.** `make stream-kafka` launched the Spark consumer, downloading the `spark-sql-kafka-0-10_2.12:3.5.0` connector jar on first run.
5. **Producer.** `scripts/kafka_producer.py` published **5 batches of 20 real reviews** (100 total) onto the topic.
6. **Output verified.** Spark consumer wrote scored output to `data/streaming_out/scored/kafka-batch-00000001.json` and `kafka-batch-00000002.json`. Each scored record carries `"source": "kafka"`, a model-derived sentiment, and a `fraud_proba`.

9.3 Why foreachBatch (not pandas UDF)

Inside `foreachBatch` the micro-batch comes back to the driver as `pandas`, the persisted joblib pipelines score it, and results are written. This avoids the `pandas-UDF broadcast-pickle` path which is fragile across Python toolchains. For very high throughput, the same logic lifts to a `pandas UDF` or a properly-broadcast joblib model.

9.4 Kafka producer note

The shipped `scripts/kafka_producer.py` calls `fh.readlines()` on `data/raw/reviews.jsonl`, which would load the full 11 GB file into memory at our scale. For the demo we pointed it at a 200-row subsample (`--source /tmp/kafka_demo_reviews.jsonl`). Streaming the source line-by-line would let it handle the full corpus without subsampling – a small follow-up.

10. Stage 5 – Serving surfaces

Five distinct surfaces consume the trained models. They share the same `models/*.joblib` and `data/processed/*.parquet` files; they differ only in transport.

10.1 FastAPI service + dashboard

`src/serve/app.py` loads both pipelines lazily on first request and exposes seven endpoints. The dashboard is a single HTML page with vanilla JS – no front-end build step.

Method	Path	What
GET	<code>/healthz</code>	Liveness probe
GET	<code>/metadata</code>	Training metrics + numeric feature columns
POST	<code>/predict</code>	Score one review (with <code>fraud_explanation</code> field)

Method	Path	What
POST	/predict/batch	Score up to 1,000 reviews
GET	/aggregates/products	Top-N from product rollup
GET	/aggregates/fraud-reviewers	Top-N suspicious reviewers
GET	/stream/recent	Latest streaming-scored rows
GET	/	Dashboard HTML

LLM fraud_explanation

Each POST /predict response includes a fraud_explanation block:

```
{
  "summary": "This review is suspicious due to its extremely low word count
              of 12 words, which is unusually brief for a genuine review...",
  "llm_generated": true,
  "risk_level": "low",
  "feature_signals": {
    "fraud_proba": 0.249855,
    "fraud_flag": 0,
    "star_rating": 1,
    "verified_purchase_int": 1,
    "body_word_count": 12,
    "exclam_count": 0,
    "promotional_wording_heuristic": false
  }
}
```

The summary is generated by Ollama llama3.2 via src/serve/fraud_explain.py. When Ollama is unreachable, a deterministic rule-based fallback fills the same field so the API contract never breaks.

10.2 Streamlit demo dashboard

streamlit_app.py (target make streamlit) is a zero-setup demo UI – it loads the joblib pipelines and the parquet aggregates **directly**, with no FastAPI server required. Five tabs:

1. **Score a review** – free-text + star slider + verified checkbox -> live sentiment + fraud_proba.
2. **Top products** – sortable table from product_agg.parquet (207,168 products), top-N slider.
3. **Suspicious reviewers** – sortable table from reviewer_agg.parquet (487,979 reviewers), sorted by fraud rate.
4. **Browse raw reviews** – paginated browser of data/raw/reviews.jsonl with optional live scoring of each row.
5. **Distributions** – Altair charts of star rating, sentiment label, and body word count over a 50,000-row sample of the test holdout.

@st.cache_resource and @st.cache_data ensure the 3 GB train parquet and the model files load once per session.

10.3 MCP server (Claude Desktop tool surface)

src/serve/mcp_server.py exposes the same scoring + aggregate access as **Claude-callable tools** through the Model Context Protocol over stdio. Built on fastmcp.

Tool	What
predict_review(review_body, star_rating, ...)	The same scorer the REST /predict uses
get_fraud_reviewers(limit)	Top suspicious reviewers from ETL aggregates
get_top_products(limit, sort_by)	Top products, sortable by review_count, fraud_rate, or product_avg_rating

Setup in Claude Desktop config (~/.Library/Application Support/Claude/claude_desktop_config.json)

```
"amazon-reviews": {
  "command": "/Library/Frameworks/Python.framework/Versions/3.13/bin/python3",
  "args": [
    "/Users/.../Large_Scale_Product_Review/src/serve/mcp_server.py"
  ]
}
```

After Claude Desktop is fully quit and reopened, the tools become available in chat:

- “Show me the top 5 most-reviewed products”
- “Score this review for fraud: ...”
- “List the most suspicious reviewers”

10.4 Ollama review auditor (autonomous agent)

src/agents/review_auditor.py is an autonomous agent that uses **Ollama tool-calling** to investigate one product. The LLM decides which tools to call iteratively, observes results, and produces a structured audit report.

The agentic loop (lines 148-179 in the file):

```
for iteration in range(8):                                # up to 8 turns
    response = ollama.chat(model="llama3.2",
                           messages=messages,
                           tools=TOOLS,                    # 4 declared tools
                           options={"temperature": 0.2})
    if not msg.tool_calls:                                  # LLM decides it has enough
        return msg.content                                  # final report
    for tc in msg.tool_calls:                                # otherwise execute each tool
        result = TOOL_FN[fn_name](**fn_args)                # call REST endpoint
        messages.append({"role": "tool", "content": result})
```

4 tools available to the agent (each hits the FastAPI server):

1. get_product_aggregate – top-level stats for a product
2. get_product_reviews – recent streaming-scored reviews

3. get_top_fraud_reviewers – suspicious reviewer leaderboard
4. score_review – score a free-form review text

Live run on real ASIN B01415QHYW:

```
[agent] calling get_product_aggregate({'product_id': 'B01415QHYW'})
[agent] calling get_product_reviews({'product_id': 'B01415QHYW'})
[agent] calling get_top_fraud_reviewers({'limit': '5'})
[agent] calling score_review(...)
```

**** PRODUCT SUMMARY ****

```
product_id      : B01415QHYW
product_category: Cell Phones and Accessories
review_count    : 42,644
avg_rating      : 4.29
fraud_rate      : 11.96%
```

**** RISK VERDICT ****

HIGH -- suspicious reviewer patterns found.

The auditor needs both **Ollama up** (ollama serve) and the **FastAPI server running** (make serve).

11. Where Ollama is and isn't used

Surface	Uses Ollama?	What for
Sentiment classification	No	sklearn LogReg(saga)
Fraud probability	No	sklearn LogReg(saga)
Streaming (Kafka or file source)	No	joblib only
Spark batch ETL	No	pure Spark
MCP server tools	No	direct joblib + parquet
FastAPI dashboard, Streamlit	No	parquet + joblib
fraud_explanation	Yes	one-shot LLM call (rule-based)
paragraph on /predict		fallback if down)
make audit PRODUCT=...	Yes	iterative tool-calling agent

Ollama is **load-bearing for the autonomous auditor** and **nice-to-have for the explanation paragraph**. Everything else runs without it.

12. ER diagram (data entities)

A high-resolution rendering is in er_diagram.png / er_diagram.svg. Summary:

Entity	What	PK	Source
RAW_REVIEW	Raw McAuley-Lab record after schema mapping	review_id	data/raw/reviews.jsonl
FEATURED_REVIEW	Cleaned + feature-engineered + labeled	review_id	data/processed/{train,test}.p
PRODUCT_AGG	Per-product rollup	product_id	data/processed/product_agg.p
REVIEWER_AGG	Per-reviewer rollup	reviewer_id	data/processed/reviewer_agg.p
SCORED_REVIEW	Streaming output	review_id	data/streaming_out/scored/*.j
MODEL_ARTIFACT	Trained joblib	path	models/*.joblib
META_JSON	Run-summary	–	models/meta.json
THRESHOLDS_JSON	Tuned threshold	–	models/thresholds.json
MLFLOW_RUN	MLflow tracked run	run_id	mlruns/
DIAGNOSTIC_REPORT	Drift / threshold / calibration / error	report_kind	reports/ml/

Key relationships:

- RAW_REVIEW -> FEATURED_REVIEW (cleaned + labeled)
- FEATURED_REVIEW -> PRODUCT_AGG, REVIEWER_AGG (rolled up)
- FEATURED_REVIEW -> MLFLOW_RUN (training input) -> MODEL_ARTIFACT -> META_JSON -> THRESHOLDS_JSON
- MODEL_ARTIFACT + new RAW_REVIEW -> SCORED_REVIEW (via streaming)
- FEATURED_REVIEW + MODEL_ARTIFACT -> DIAGNOSTIC_REPORT (offline)

13. Tests

tests/test_pipeline.py exercises the code paths most likely to fail silently:

- Text cleaning (URL, HTML, punctuation stripping).
- Star-rating-to-sentiment label mapping (parametrised over all 6 cases).
- The serving-time feature enrichment (every numeric feature the fraud model expects must be present after enrichment).
- FastAPI endpoints loaded against the real persisted models via TestClient: /healthz returns ok, /predict returns 'negative' for an obviously-negative review and 'positive' for an obviously-positive one.

```
$ make test
10+ tests passing
```

14. Complete command cheat sheet

```
# === Setup ===
make install                                # pip install -r requirements.txt

# === Data ===
# (one-time) download McAuley-Lab category file (8.7 GB)
curl -L -o /path/to/Cell_Phones_and_Accessories.jsonl \
  "https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023/resolve/main/raw/re

# (one-time) convert source -> project schema (~3.5 min)
python3 -m src.ingest.import_amazon_real \
  --src /path/to/Cell_Phones_and_Accessories.jsonl \
  --out data/raw/reviews.jsonl

# === Pipeline ===
SPARK_DRIVER_MEM=8g make etl                # ~24 min on 20.5M rows
make train                                  # ~5 hours on 16.4M rows
make stream-once                           # one-shot streaming pass

# === Serving ===
make serve                                  # FastAPI on http://127.0.0.1:8000/
make streamlit                             # Streamlit on http://127.0.0.1:8501/
make mlflow-ui                             # MLflow on http://127.0.0.1:5000/

# === Streaming (Kafka path) ===
export JAVA_HOME=$(brew --prefix openjdk@17)/libexec/openjdk.jdk/Contents/Home
make kafka-start                           # broker (KRaft mode)
make kafka-topic-create                    # 'reviews' topic
make stream-kafka                          # term 1: Spark consumer
python3 scripts/kafka_producer.py          # term 2: producer

# === Diagnostics ===
python3 -m src.train.drift_monitor          # train vs test drift
python3 -m src.train.threshold_tuning       # threshold sweep + thresholds.json
python3 -m src.train.calibration_report     # raw vs sigmoid vs isotonic
python3 -m src.train.error_analysis         # error samples
python3 -m src.train.baselines              # baseline comparison (heavy)
python3 -m src.train.ablation_study         # leakage ablation (heavy)

# === LLM-backed ===
make mcp                                    # MCP server (Claude Desktop)
make audit PRODUCT=B01415QHYW              # autonomous agent

# === Tests ===
make test                                   # unit + integration

# === Cleanup ===
```

```
make clean
make kafka-stop
```

```
# wipe data/ + models/ + mlruns/
# stop the broker
```

15. Demo cheat sheet

15.1 URLs (browser-openable)

Service	URL	What to show
Streamlit	http://127.0.0.1:8501/	Friendliest UI; 5 tabs
FastAPI dashboard	http://127.0.0.1:8000/	Production dashboard with LLM fraud explanation
MLflow	http://127.0.0.1:5000/	Tracked runs + metrics + artifacts
Spark Web UI	http://127.0.0.1:4040/Streaming	Live/streaming query (only while consumer runs)
Ollama	http://127.0.0.1:11434/	Local LLM (no UI, just API)

15.2 Recommended 8-minute flow

1. **Streamlit** – show sidebar metrics, score a 1-star and 5-star review, browse top products. Friendliest opener.
2. **FastAPI dashboard** – same data, “production-style” UI. Score a review and show the LLM fraud_explanation panel.
3. **MLflow** – click into the fraud run, show metrics + artifacts.
4. **Spark Web UI** – click Structured Streaming tab; show that the Kafka consumer is live.
5. **Push to Kafka** – run the producer; flip back to FastAPI’s /stream/recent to show new scored rows with "source":"kafka".
6. **Autonomous agent** – run `make audit PRODUCT=B01415QHYW` in a terminal. Show [agent] calling ... trace and the structured RISK VERDICT.
7. **Claude Desktop with MCP** – ask Claude “What are the top 5 most fraudulent products?” – show that Claude is calling our get_top_products MCP tool.

15.3 Live-producing trick

To make the Spark Jobs UI tab fill with new entries during the demo, run a side terminal loop:

```
while true; do
  python3 scripts/kafka_producer.py --source /tmp/kafka_demo_reviews.jsonl \
    --batch-size 10 --n-batches 1 --sleep 0
  sleep 6
done
```

16. Honest caveats – read before quoting numbers

1. **Single product category.** All 20.8 M reviews are Cell Phones and Accessories. The `product_category` dimension is single-valued; category-based features add no signal. Loading 5-10 categories side-by-side is the immediate next improvement.
 2. **`total_votes == helpful_votes`.** The 2023 source dropped unhelpful counts. Any feature that uses `helpful / total` ratio is uniformly 1.0 (or 0/0) and is not informative.
 3. **Sentiment LogReg did not fully converge.** `saga` hit `max_iter=100` before `tol`; weights are usable but suboptimal. Bumping to 300 iterations would converge but triple training time.
 4. **GBT and heavy RF were swapped out.** The proposal mentions Gradient Boosted Trees as the fraud model. We trained two scalable substitutes (`SGDClassifier` and `LogisticRegression(saga)`) because GBT is single-threaded and would not finish on 16 M rows in any reasonable wall-clock budget on a laptop. The structural design is unchanged. **If your rubric specifically requires GBT, this would need either a real cluster or a sub-sample for the fraud trainer.**
 5. **`baselines.py` and `ablation_study.py` not run on full data.** Both retrain models per call. `ablation_study` uses the original `GradientBoostingClassifier` (single-threaded), so running it on the full 16 M corpus would take days. Both are tractable on a 1 M subsample.
 6. **Why fraud F1 is low even though AUC is 0.845 – this is not a contradiction.** ROC-AUC measures how well the model *ranks* reviews, independent of any threshold; 0.845 means a fraud review scores higher than a clean review 84.5% of the time. F1 at the default 0.5 threshold is poor because (a) the class is imbalanced (3.57% positives) and (b) the ETL added intentional label noise. The threshold-tuning study found 0.80 as the better operating point.
-

17. Where this would go next

- **Multi-category training.** Concatenate 5-10 category JSONLs to break the single-category pall and let category-based features add signal.
- **Wire the tuned threshold into serving.** `models/thresholds.json` now holds `best_f1_threshold = 0.80`; have FastAPI honor it instead of the hard-coded 0.5 in the `fraud_flag` decision.
- **Wire calibration into serving.** Section 8.3 found isotonic calibration cuts Brier ~5x with no ROC-AUC change. Emit isotonic-calibrated `fraud_proba` from `/predict`.
- **Stream the Kafka producer source.** `scripts/kafka_producer.py` currently calls `readlines()`, which OOMs on the full 11 GB JSONL. Switch to line-by-line read so the producer can replay the full corpus into Kafka at any rate.
- **Hand-labeled fraud sample.** Build a small (~1,000-row) human-labeled validation set so the AUC isn't measured against a noisy weak-label proxy.
- **Run `baselines.py` and `ablation_study.py` on a 1 M subsample** to ground the chosen architecture against simpler / alternative classifiers.
- **Promote models through MLflow Model Registry.** Stage -> production promotion, FastAPI loads by stage rather than by file path.

- **Containerize.** Dockerfile + docker-compose for Spark + MLflow + Kafka + FastAPI + Ollama so the project deploys on any host with a single command.
- **Feature store.** Real reviewer / product history at serving time instead of neutral defaults – closes the train/serve skew gap on the behavioral features.

Appendix A – Code changes

File	Change	Commit
src/common/spark.py	Driver memory 2g -> 8g (env-overridable), shuffle partitions 8 -> 200, AQE on, Kryo serializer, larger maxResultSize.	8e2c65c
src/train/train.py	Column-subset parquet read + dtype downcast; sentiment LogReg switched to saga; replaced GradientBoostingClassifier and heavy RandomForestClassifier with SGDClassifier(log_loss) and LogisticRegression(saga); tightened TF-IDF (min_df=10, fraud max_features=15_000).	10b63b7
streamlit_app.py (new) + Makefile	Single-file Streamlit dashboard + make streamlit target.	a9431fc
PROJECT_REPORT.md (new) + PROJECT_REPORT.pdf	Build report.	1e0f013, d267887, then expanded
README.md	Refreshed for full-scale run + LLM/agent surfaces.	9917974
src/ingest/import_amazon_reviews.py	Standardizing adapter: McAuley-Lab schema -> project schema.	9917974
data/raw/reviews.jsonl	Symlink to the converted 11 GB JSONL (avoids duplicating data).	(not in git – local)

Appendix B – Commits pushed to origin/main (this session)

9917974 docs: README reflects full real-data run + LLM/agent surfaces
d267887 docs: report covers full-feature run (Kafka + diagnostics + agents)
1e0f013 docs: full real-data run report (PROJECT_REPORT.md + .pdf)

a9431fc feat: Streamlit demo dashboard
10b63b7 refactor(train): scale trainer to 20M-row corpus
8e2c65c chore(spark): tune SparkSession for full-scale ETL

Browse: https://github.com/Paruchuri-Rajesh/Large_Scale_Product-Review/commits/main

Appendix C – File inventory at end of run

Source code (in git):

src/ingest/import_amazon_real.py	← schema adapter
src/ingest/generate_sample.py	← synthetic generator (dev)
src/etl/batch_etl.py	← Spark batch ETL
src/train/train.py	← main trainer
src/train/{features, registry, evaluate, utils}.py	
src/train/{baselines, threshold_tuning, error_analysis, drift_monitor, ablation_study, calibration_report}.py	
src/stream/score_stream.py	← Kafka consumer
src/stream/score_stream_kafka.py	← Kafka consumer (alt)
src/serve/app.py	← FastAPI
src/serve/fraud_explain.py	← Ollama + fallback
src/serve/mcp_server.py	← MCP server
src/serve/templates/index.html	
src/serve/static/{app.js, style.css}	
src/agents/review_auditor.py	← Ollama tool-calling agent
src/common/{config, schema, spark, text}.py	
scripts/{feed_stream.py, kafka_producer.py, run_pipeline.sh, build_report.py}	
streamlit_app.py	← Streamlit demo
tests/test_pipeline.py	
Makefile / requirements.txt / .gitignore	
README.md / PROJECT_REPORT.{md, pdf}	
architecture.{png, svg} / er_diagram.{png, svg}	

Generated artifacts (in .gitignore):

data/raw/reviews.jsonl	← 11 GB project-schema JSONL
data/processed/train.parquet	← 3.0 GB
data/processed/test.parquet	← 779 MB
data/processed/product_agg.parquet	← 17 MB
data/processed/reviewer_agg.parquet	← 325 MB
data/streaming_out/scored/kafka-batch-*.json	
models/sentiment_pipeline.joblib	← 3 MB
models/fraud_pipeline.joblib	← 687 KB
models/meta.json	
models/thresholds.json	← best_f1_threshold = 0.80
mlruns/	← MLflow tracking
reports/ml/drift_report.json	
reports/ml/threshold_study.csv	
reports/ml/fraud_calibration_{report, bins, summary}.{csv, json}	

reports/ml/{sentiment,fraud}_error_samples.csv (188 + 85 MB)

Appendix D – Glossary

- **Adapter** (`import_amazon_real.py`) – script that converts McAuley-Lab JSONL into project schema, streaming line-by-line.
- **AQE** – Adaptive Query Execution; Spark feature that re-plans shuffles based on runtime statistics.
- **ASIN** – Amazon Standard Identification Number; used as `product_id`.
- **Brier score** – mean squared error of probabilistic predictions; lower is better-calibrated.
- **ECE** – Expected Calibration Error; how far predicted probabilities deviate from observed frequencies.
- **ETL** – Extract, Transform, Load – the pattern of reading from a source, transforming it, and persisting to a queryable store.
- **fastmcp** – Python MCP framework used by `src/serve/mcp_server.py`.
- **foreachBatch** – Spark Structured Streaming sink that hands each micro-batch to a Python function as a `DataFrame`.
- **Heuristic / weak label** – label generated by rules rather than human ground truth; used here for fraud.
- **Isotonic calibration** – non-parametric monotone mapping of raw probabilities to better-calibrated ones.
- **KRaft** – Kafka’s “Raft” mode that removes the Zookeeper dependency.
- **MCP** – Model Context Protocol; lets external apps (e.g. Claude Desktop) call your tools over JSON-RPC stdio.
- **MLflow** – experiment tracking server; stores params, metrics, artifacts.
- **saga solver** – Stochastic Average Gradient Augmented; the scikit-learn LR solver designed for large sparse problems.
- **Window function** – Spark SQL operation that computes aggregates over a partition (e.g. all rows with the same `reviewer_id`).